

Лабораторна робота №1. Надійний typed LLM-клієнт

25 серпня 2026 р.

Мета

Побудувати клієнт до мовної моделі, який перетворює недовірений текст на валідований типізований об'єкт або на контрольовану відмову, і жодного разу не повертає неперевірений результат як успішний.

Робота спирається на лекції 1–5. Аудиторний час — 3 години, самостійна робота — 6 годин. Усі автоматичні перевірки мають проходити без мережі та без ключа API.

Теоретичні відомості

Мовна модель повертає розподіл над продовженнями, а не значення функції. Та сама пара «інструкція + запит» може дати різні виходи, і причина не лише в семплінгу: на результат впливають версія моделі, склад і порядок контексту, а на GPU ще й пакетування запитів разом із неасоціативністю операцій з рухомою комою. Тому текст відповіді не є інтерфейсом, а межа між моделлю і кодом описується контрактом.

- **Три рівні гарантії формату.** Інструкція в промпті «відповідай у JSON» не гарантує нічого: вона змінює ймовірності, але не забороняє невалідний вихід. Валідація після генерації гарантує, що некоректний результат не пройде далі, ціною повторних викликів. **Обмежене декодування** (constrained decoding) будує граматику допустимих продовжень і на кожному кроці забороняє токени, що виводять за межі схеми; синтаксична валідність тоді гарантована самою процедурою генерації. Провайдерські API реалізують це під назвою `strict mode`, локально ту саму роль виконують GBNF-граматики в `llama.cpp` або бібліотека `Outlines`.
- **Схема описує форму, а не зміст.** Модель `Pydantic` породжує JSON Schema, у якій важливі три речі: перелік `required`, типи полів і заборона зайвих полів (`additionalProperties: false`). Остання не косметична: без неї модель домальовує поля, яких немає в контракті, і

вони мовчки проходять далі. Але навіть строга схема гарантує лише те, що поле `date` існує і є рядком, а не те, що дата правильна. Доменні правила перевіряє код, і ніхто інший за нього цього не зробить.

- **Чотири класи помилок вимагають чотирьох різних реакцій.** *Транспортна* (429, 500, 504, 529) каже, що виклик не дійшов, і про зміст нічого не каже — її повторюють із затримкою. *Схемна* означає, що відповідь не лягла в контракт — повторюють не більше двох разів, передаючи моделі текст помилки валідатора. *Доменна* означає, що відповідь валідна за формою і неприйнятна за правилами задачі — повтор не допоможе, результат іде користувачеві. *Відмова моделі* є законною відповіддю, а не збоєм, і обробляється окремою гілкою.
- **Повтор без обмежень перетворює збій на аварію.** Політика повтору складається з трьох частин: обмеженого числа спроб, експоненційного зростання затримки та випадкової складової (джитера), без якої всі клієнти синхронно б'ють у сервіс в одну мить. Поверх цього ставлять таймаут на окремий виклик і бюджет часу на весь запит, інакше повільна відповідь з'їдає бюджет користувача мовчки.
- **Ідемпотентність робить повтор безпечним.** Ключ операції обчислюється до виклику з тих даних, що визначають операцію, і зберігається разом із результатом. Тоді повторений запит повертає збережену відповідь, а не створює другий запис. Без ключа будь-який ретрай на дії зі станом подвоює наслідок.

Корисні посилання для ознайомлення:

- Pydantic: [JSON Schema](#) та [Strict Mode](#);
- OpenAI: [Structured Outputs](#);
- [JSON Schema Validation, draft 2020-12](#);
- llama.cpp: [GBNF Grammars](#);
- M. Brooker, [Exponential Backoff And Jitter](#), AWS Architecture Blog.

Порядок виконання

Позначка *ауд.* означає, що крок виконується на занятті, *сам.* — самостійно в межах СРС. Аудиторна частина розрахована рівно на 3 години.

1. **Підготувати середовище** (30 хв, ауд.). Розгорнути локальну модель через `Ollama` або `llama.cpp` (3–8В, 4-бітний квант) і окремо режим `stub`, який відтворює записані відповіді з файлу.

Перевірка перед наступним кроком: усі тести проходять у режимі `stub` без мережі. Якщо для тестів потрібна жива модель — контракт спроектовано неправильно.

2. **Описати контракт результату** (40 хв, ауд.) як модель Pydantic: не менше п'яти полів, серед них хоча б одне числове з діапазоном, одне з перелічуваного типу і одне необов'язкове. Заборонити додаткові поля. Вивести з моделі JSON Schema і додати її до звіту.
Перевірка: у схемі є `required` з усіма полями і `additionalProperties: false`. Вручну підсунути об'єкт із зайвим полем — має впасти валідація, а не пройти далі.
3. **Реалізувати розбір з рівно однією спробою відновлення** (40 хв, ауд.): у разі невалідного JSON повторний запит містить текст помилки валідації. Друга невдала спроба завершується контрольованою відмовою.
Перевірка: у логах видно, що друга спроба отримала саме текст помилки, а не той самий промпт.
4. **Реалізувати класифікацію помилок за чотирма класами** (30 хв, ауд.). Для кожного класу повернути типізований об'єкт відмови з полями: клас, повідомлення, чи доречний повтор.
Перевірка: кожен клас відтворюється окремим випадком із набору, і для кожного видно рішення про повтор.
5. **Додати механізми надійності** (40 хв, ауд.): повтор з експоненційним зростанням затримки та джитером, таймаут на окремий виклик і бюджет часу на весь запит.
Перевірка: штучно сповільнити `stub` і показати в логах, що обробку зупинив саме бюджет, а не виняток.
6. **Реалізувати ідемпотентність** (30 хв, сам.): обчислити ключ операції до виклику, зберігати результати у файлі або SQLite. Повторний запит з тим самим ключем не має створювати другий запис.
7. **Підготувати набір випадків і тести** (60 хв, сам.): не менш ніж 30 синтетичних випадків, серед яких коректні, з відсутнім полем, з полем неправильного типу, із зайвим полем, з порушенням доменного правила, з обірваним JSON і з відмовою моделі. Написати не менше 12 тестів `pytest`.
Перевірка: у наборі є обірваний JSON і відмова моделі — саме вони найчастіше не покриті.
8. **Виміряти й оформити результат** (40 хв, сам.). На цьому наборі виміряти: частку валідних за схемою відповідей, частку успішних відновлень, частку контрольованих відмов, середню кількість токенів на запит і затримку р50 та р95. Звести в одну таблицю.
9. **Здати роботу:** код і README з однією командою запуску, набір випадків, файл результатів, таблицю метрик і висновок обсягом до 150 слів про те, який клас помилок виявився найчастішим і чому.

Типова помилка порядку: писати тести після коду. Набір випадків із кроку 7 корисніше скласти одразу після кроку 2 — тоді контракт перевіряється на реальних поламаних даних.

Контрольні запитання

1. Чому температура 0 не гарантує однакового виходу на однаковому вході?
2. Що саме гарантує обмежене декодування і чого воно не гарантує?
3. Навіщо в схемі `additionalProperties: false`, якщо зайві поля можна просто ігнорувати?
4. Чим доменна помилка відрізняється від помилки схеми і чому реакція на них різна?
5. У яких випадках повторний виклик робить гірше, ніж відмова?
6. Що станеться, якщо у політиці повтору немає випадкової складової?
7. Як обчислити ключ ідемпотентності для запиту, що містить довільний текст користувача?
8. Чому одна спроба відновлення, а не три?
9. Які метрики покажуть, що промпт погіршився після зміни, а які цього не покажуть?
10. Чому потокова передача (streaming) насамперед змінює час до першого токена і сприйману затримку, а не обсяг генерації, і за яких умов вона все ж змінює повний час відповіді?

Література

1. Willard B., Louf R. [Efficient Guided Generation for Large Language Models](#). arXiv:2307.09702, 2023.
2. Xiao T., Zhu J. [Foundations of Large Language Models](#). arXiv:2501.09223, 2025. Розділ 5.
3. Jurafsky D., Martin J. H. [Speech and Language Processing](#), 3rd ed. draft, 2026. Розділ 7.
4. Anthropic. [API errors and retry behaviour](#).